

WEEK-3

Task 3:

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
root@LAPTOP-RJFD8EBT:~/OSSP/src# touch task3.c
root@LAPTOP-RJFD8EBT:~/OSSP/src# ls -l task3.c
-rw-r--r-- 1 root root 0 Aug 23 18:18 task3.c
root@LAPTOP-RJFD8EBT:~/OSSP/src# nano task3.c
root@LAPTOP-RJFD8EBT:~/OSSP/src# cat task3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Before fork()\n");
    printf("Original Process PID = %d\n", getpid());
    printf("Original Process PPID = %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("----- CHILD PROCESS ----- \n");
        printf("Child PID = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());
        printf("Child State: Running\n");

        printf("Child is sleeping for 5 seconds...\n");
        sleep(5);

        printf("Child State: Running\n");
        printf("Child exiting...\n");

        exit(0);
    }

    else {
        // Parent process
        printf("----- PARENT PROCESS ----- \n");
        printf("Parent PID = %d\n", getpid());
        printf("Parent PPID = %d\n", getppid());
        printf("Parent State: Running\n");

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent State: Running\n");
        printf("Parent exiting...\n");
    }

    return 0;
}
```

```
root@LAPTOP-RJFD8EBT:~/OSSP/src# gcc task3.c -o ../bin/task3
root@LAPTOP-RJFD8EBT:~/OSSP/src# ls -l ../bin/task3
-rwxr-xr-x 1 root root 16296 Aug 23 18:19 ../bin/task3
root@LAPTOP-RJFD8EBT:~/OSSP/src# ../bin/task3
Before fork()
Original Process PID = 892
Original Process PPID = 318

----- PARENT PROCESS -----
Parent PID = 892
Parent PPID = 318
----- CHILD PROCESS -----
Parent State: Running
Parent is waiting for child...
Child PID = 893
Child PPID = 892
Child State: Running
Child is sleeping for 5 seconds...
Child State: Running
Child exiting...
Child terminated.
Parent State: Running
Parent exiting...
root@LAPTOP-RJFD8EBT:~/OSSP/src# ../bin/task3
Before fork()
Original Process PID = 897
Original Process PPID = 318
```

```

----- PARENT PROCESS -----
Parent PID = 892
Parent PPID = 318
----- CHILD PROCESS -----
Parent State: Running
Parent is waiting for child...
Child PID = 893
Child PPID = 892
Child State: Running
Child is sleeping for 5 seconds...
Child State: Running
Child exiting...
Child terminated.
Parent State: Running
Parent exiting...
root@LAPTOP-RJFD8EBT:~/OSSP/src# ./bin/task3
Before fork()
Original Process PID = 897
Original Process PPID = 318

----- PARENT PROCESS -----
Parent PID = 897
Parent PPID = 318
Parent State: Running
Parent is waiting for child...
----- CHILD PROCESS -----
Child PID = 898
Child PPID = 897
Child State: Running
Child is sleeping for 5 seconds...
Child State: Running
Child exiting...
Child terminated.
Parent State: Running
Parent exiting...
root@LAPTOP-RJFD8EBT:~/OSSP/src# ps -ef | grep task3
root      900      318  0 18:20 pts/0    00:00:00 grep --color=auto task3
root@LAPTOP-RJFD8EBT:~/OSSP/src# ps -eo pid,ppid,state,cmd | grep task3
      902      318  S  grep --color=auto task3
root@LAPTOP-RJFD8EBT:~/OSSP/src#

```

Task 4:

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```

root@LAPTOP-RJFD8EBT:~/OSSP# touch task4.c
root@LAPTOP-RJFD8EBT:~/OSSP# ls -l task4.c
-rw-r--r-- 1 root root 0 Aug 23 18:49 task4.c
root@LAPTOP-RJFD8EBT:~/OSSP# nano task4.c
root@LAPTOP-RJFD8EBT:~/OSSP# cat task4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process starting...\n");
    fflush(stdout);

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("----- CHILD PROCESS ----- \n");
        printf("Child PID = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());
        printf("Child State: Running\n");
        fflush(stdout);

        printf("Child is sleeping for 30 seconds...\n");
        fflush(stdout);

        sleep(30);

        printf("Child State: Running\n");
        printf("Child terminating...\n");
        fflush(stdout);

        exit(0);
    }
    else {
        printf("----- PARENT PROCESS ----- \n");
        printf("Parent PID = %d\n", getpid());
        printf("Parent PPID = %d\n", getppid());
        printf("Parent State: Running\n");
        printf("Parent is waiting for child...\n");
        fflush(stdout);

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent State: Running\n");
        printf("Parent terminating...\n");
        fflush(stdout);

        return 0;
    }
}

root@LAPTOP-RJFD8EBT:~/OSSP# gcc task4.c -o ./bin/task4
/usr/bin/ld: cannot open output file ./bin/task4: No such file or directory
collect2: error: ld returned 1 exit status
root@LAPTOP-RJFD8EBT:~/OSSP# ls -l ./bin/task4
ls: cannot access './bin/task4': No such file or directory
root@LAPTOP-RJFD8EBT:~/OSSP# cd ~/OSSP/src
root@LAPTOP-RJFD8EBT:~/OSSP/src# gcc task4.c -o ./bin/task4
root@LAPTOP-RJFD8EBT:~/OSSP/src# ls -l ./bin/task4
-rwxr-xr-x 1 root root 16296 Aug 23 18:52 ./bin/task4
root@LAPTOP-RJFD8EBT:~/OSSP/src# cd ~/OSSP
root@LAPTOP-RJFD8EBT:~/OSSP# ./bin/task4
-bash: ./bin/task4: No such file or directory
root@LAPTOP-RJFD8EBT:~/OSSP# ls -l bin/task4
-rwxr-xr-x 1 root root 16296 Aug 23 18:52 bin/task4
root@LAPTOP-RJFD8EBT:~/OSSP# file bin/task4
bin/task4: ELF 64-bit LSB pie executable, x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=1e218865befe1fdb59cf9115c578522d6728552, for GNU/Linux 3.2.0, not stripped
root@LAPTOP-RJFD8EBT:~/OSSP# ./bin/task4
Parent process starting...
Parent process started.
Child process started.
Parent PID = 1166
Child PID = 1167
Parent is waiting for child...
Child PID = 1167
Child PPID = 1166
Child is running...
Child terminating...
Child terminated.
Parent terminating...
root@LAPTOP-RJFD8EBT:~/OSSP#

```